

Software Requirements Specification (SRS)

Group 5

Team members:

Punam :23524202064

Rifat: 23524202096

Mehrab: 23524202112

Orce: 23524202136

Project: Ghuri-Phiri – A Smart Tourist Guide & Budget Planner

1. Introduction

1.1 Purpose

This SRS document aims to offer an in-depth description of the web application "Ghuri-Phiri". It specifies the system's functional and non-functional requirements and acts as a reference for development, testing, and deployment. Through this document, the development team (Punam, Rifat, and Orce and Mehrab) and the lab instructors come to a mutual and clear understanding of the project's scope.

1.2 Intended Audience

This document is intended for:

- **Database (Punam):** SQL database architecture.
- **Backend Developer (Mehrab):** To implement backend logic, API integrations
- **Testers (Rifat):** To build and verify system functionality against requirements.
- **Project Manager and Frontend Developer (ORCE):** To monitor milestones and ensure documentation alignment. To build the frontend components.
- **Academic Evaluators:** To assess the technical depth and planning of the SDP-I project.

1.3 Intended Use

The SRS will be used throughout the development lifecycle (March 25 – May 15) to maintain consistency. It will be the primary reference during the Quality Assurance (QA) phase to ensure all user stories are fulfilled.

1.4 Product Scope

Ghuri-Phiri is a creative travel assistant dedicated to Bangladesh only. It has the following objectives:

- Gathering information about tourist spots for all 8 divisions in one platform.
- Being able to dynamically calculate travel costs (Transport, Hotel, Food).
- Giving Live Weather Alerts through external API integration.
- Making the discovering process easier through Global Smart Filters (Category/Budget).
- Making the use of Google Maps more convenient through integration.

1.5 Risk Definitions

- **API Dependency Risk:** If the Google Maps or OpenWeatherMap APIs were to fail, there would be no way of offering the real-time data features.
- **Data Obsolescence:** If the travel costs (bus/hotel rates) in the SQL database are not refreshed then users will be getting the wrong budget estimations.
- **Time Constraint Risk:** A hectic timetable may cause a delay in development; therefore the use of modular design is a must to get this risk under control.

2. Overall Description

2.1 User Classes and Characteristics

- **General turistas:** People who visit places often and need simple ways to find their way around and plan their spending quickly and easily.
- **Researchers/students:** People who want to get details about the history and culture of the heritage sites in an elaborate manner.
- **Administrators:** Users with a certain degree of technical skills who are responsible for running the database, changing the costs, and adding new destinations.

2.2 User Needs

The users want a single platform that not only saves them the effort of going to several websites but also gives them all the necessary information for them to make the decision. They want to be guided on where to go according to their budget, be aware of the price, and find out what the weather conditions are like at that particular place at the moment.

2.3 Operating Environment

- **Platform:** Modern Web Browsers (Chrome, Firefox, Safari, Edge).
- **Architecture:** Client-Server model.
- **Frontend:** HTML5, Tailwind CSS, JavaScript.
- **Backend:** Node.js/Express.js.
- **Database:** MySQL.

2.4 Constraints

- **Hardware:** Should work on low-end mobile devices as well as high-end desktops (Responsive Design).
- **Deadline:** The complete and documented project should be ready by May 15.
- **Regulatory:** Implementing web security best practices for the Admin Panel login is a must.

2.5 Assumptions

- User will have a stable internet connection with no interruptions.
- Third-party APIs (Google/OpenWeather) will still be available and allow free-tier usage for development purposes.

3. Requirements

3.1 Functional Requirements (User Story Format)

ID	User Story	Confirmation / Acceptance Criteria
FR1	As a traveler, I want to employ a global filter to find "Hill" or "Beach" spots throughout all divisions so that I am able to discover places based on what I like instead of the location alone.	The system provides a grid of results that are tagged with the category matched across all 8 divisions.
FR2	As someone who always watches their budget, I want to find out the approximate total expense for a 2-day vacation, so I will be able to figure out whether a place is within my means or not.	The "Place Details" page shows a computed grand total, which is based on the latest SQL data of transport, food, and lodging.

FR3	Being a traveler who plans ahead, I wish to have a live-weather badge on destination page so that I can avoid traveling in case of storms or heavy rainfall.	One of the things a widget could do is show the present temperature along with a weather icon (like a sun, rain clouds, etc.) that are retrieved from OpenWeatherMap API.
FR4	I'd like to be able to click on the map link and then the map viewer opens up with the destination marked. Also, please provide me the option of getting a route from my current location to the destination.	A map from Google that is embedded or even just a "View on Maps" button will open the exact location in a new tab/window.
FR5	As an admin, I want to be able to change the "Bus Fare" or "Hotel Rate" via a secure dashboard to provide users with the latest and most accurate budget estimates at all times.	The modifications done in the admin panel reflect in the frontend right after refreshing the page.

3.2 Non-Functional Requirements

- **Performance:** - The site should respond with the homepage and search results within 3.0 seconds.
 - Database queries used for filtering must be designed to yield rapid response times.
- **Security:**
 - Security is paramount. The Admin Dashboard's login system, whether session- or token-based, must be robust.
- **Quality (Usability):**
 - The UI should use a consistent color pallet and typography (using Tailwind CSS).
 - The design be "Mobile First" to ensure it is usable on smartphones.
- **Availability:**
 - Essential data (text/history) must still be accessible albeit a failure of the weather API (Graceful degradation).